DSA03-NATURAL LANGUAGE PROCESSING ASSIGNMENT-1

ASSIGNMENT 1 – CO1

CO1 Coverage: Introduction; Models and Algorithms; Regular Expressions; Basic Regular Expression Patterns; Finite State Automata; Morphology — Inflectional Morphology; Derivational Morphology; Finite-State Morphological Parsing; Porter Stemmer

Nature of Assignment: Individual
Duration: 1 Week
Total Marks: 50
Submission Date:04/11/2026
NAME: Seene Ishaq Ahamed S
REGNO: 192225064

Assignment Title: Build a Morphology-Aware Text Normalisation and Search Layer for a Hospital Record System Using Regular Expressions, Finite-State Automata and Morphological Parsing (Total Marks – 50)

Duration: 1 week

Scenario — Text Normalisation and Search for a Hospital Record System

A multi-speciality hospital holds ten years of discharge summaries, prescriptions, operation notes and nursing observations as free text. Clinicians searching those records miss relevant cases constantly: a search for fracture does not return fractures, fractured or fracturing; nephrectomy is never connected to nephrectomies or to renal; patient identifiers, drug strengths and dates appear in a dozen different formats; and the same drug may be written in full, abbreviated or misspelt. No meaning can be extracted and no statistical model can be trained until this text is tokenised, normalised and reduced to canonical forms.

You are the language-engineering team. Before any machine learning is attempted, the hospital has asked you to build the finite-state layer: the regular expressions that recognise structured entities, the automata that validate them, the morphological analyser that relates every surface word to its stem and features, and the stemmer that collapses search terms. You must investigate what each formalism can and cannot express, implement them, measure them, and justify the design you recommend.

Planning data and constraints

- Corpus: at least 500 de-identified clinical records (real or synthetic, but documented), totalling at least 25,000 word tokens and covering at least four record types.
- Pattern inventory: at least 12 structured entity types to be recognised by regular expression — patient identifier, admission and discharge date, drug name with strength, dosage frequency, laboratory value with unit, blood pressure, temperature, ICD code, ward and bed, contact number, age, and duration of symptom.
- Automata: at least four finite-state automata, each with at least six states, including one entity format that cannot be recognised conveniently without non-determinism.
- Morphological resources (build and document your own): a lexicon of at least 300 stems, at least 20 inflectional affixes and at least 15 derivational affixes, with irregular forms listed separately.
- Evaluation data: at least 40 clinician queries with relevance judgements over the corpus, and at least 200 word forms hand-annotated with their stem and features.

- Any manual effort (pattern writing, lexicon building, annotation, testing) must be accounted for in your effort and cost discussion.

Design Challenge (Day 4):

The system must now ingest notes captured by optical character recognition from scanned handwritten forms, and notes typed in Romanised regional language mixed with English. Re evaluate your original patterns, automata and analyser and decide whether to retain, modify or redesign them.

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis 3

Marks Guiding Enquiry

Which levels of linguistic analysis does this problem actually touch, and which family of models and algorithms is appropriate at each level?

Investigate and interpret:

- Describe the levels of linguistic analysis — phonology, morphology, syntax, semantics, pragmatics and discourse — and state, with an example drawn from your own corpus, which levels the record-search problem requires and which it can safely ignore.
- Distinguish the three model families used throughout this unit: state machines, formal rule systems and probabilistic models. Name one task in the hospital problem that each family suits best and say why the other two would be the wrong choice for it.
- Explain why ambiguity is the central problem of natural language processing, and identify at least four distinct ambiguities present in your corpus — lexical, morphological, segmentation and abbreviation ambiguity.
- Identify where search appears in this unit (state-space search in automata, backtracking in matching) and state the resources each consumes: time, memory, annotated data and expert effort.
- Set out, as a labelled diagram, the pipeline you will build in the remaining tasks, naming the input, the output and the formalism of every stage.

Evidence Expected

A reasoned problem framing that maps the hospital task onto levels of analysis, model families and a concrete, staged pipeline.

TASK 2 – Basic Regular Expression Patterns 4 Marks

Guiding Enquiry

What can the basic regular expression operators express, and how are they combined into patterns that match real clinical strings exactly?

Investigate and interpret:

- Define the basic operators — concatenation, disjunction, Kleene star and plus, optionality, character classes, ranges, negated classes, anchors and counters — and give one minimal clinical example of each.
- Write and document a regular expression for each of the twelve entity types in your pattern inventory, stating for every pattern both what it must match and what it must not.
- Demonstrate greedy and non-greedy matching on a clinical sentence containing two dosage expressions, and explain precisely why the two quantifiers return different spans.
- Explain operator precedence and grouping, and show with a worked example how a misplaced disjunction silently changes the language the expression denotes.
- Evaluate every pattern over the corpus, reporting true positives, false positives and false negatives, and name the specific linguistic cause of each error class.

Evidence Expected

A documented pattern inventory with precise match specifications and a per-pattern error analysis carried out on real data.

TASK 3 – Regular Expressions for Tokenisation and Normalisation 4

Marks Guiding Enquiry

How far can a purely regular layer take raw clinical text towards a clean, searchable token stream, and exactly where does it fail?

Investigate and interpret:

- Specify your tokenizer as a single regular expression, defining precisely what counts as a token — hyphenated terms, abbreviations containing full stops, decimal values, units written against numbers — and justify each decision.
- Apply maximal-munch (longest-match) tokenisation and show on at least three examples how the result differs from a naive split on whitespace.
- Build substitution rules that fold case, expand at least thirty clinical abbreviations and canonicalise date and unit formats; show the complete before-and-after text of one full record.
- Identify at least three normalisation decisions that are irreversible because information is destroyed, and argue when that loss is acceptable and when it is not.
- Give one requirement of the hospital task that no regular expression can express, and argue informally why — for example, matching balanced parentheses in a nested clinical note.

Evidence Expected

A working regular-expression tokenizer and normaliser with worked examples and a clear statement of the limits of regularity.

TASK 4 – Finite State Automata: Design, Non-determinism and Equivalence 5 Marks Guiding Enquiry

How do automata recognise exactly the same languages as regular expressions, and what does removing non-determinism cost?

Investigate and interpret:

- Define a finite automaton formally as the five-tuple $(Q, \Sigma, \delta, q_0, F)$, then design a deterministic automaton for at least four validators from your pattern inventory, giving both the transition diagram and the transition table for each.
- Design a non-deterministic automaton with ε-transitions for one entity whose format has optional parts, and trace its behaviour on one accepted and one rejected string, showing the set of states live at every step.
- Convert that NFA to a DFA by subset construction, showing the complete construction table, then minimise the DFA and report the state counts before and after.
- Apply Thompson's construction to turn one Task 2 regular expression into an automaton, and derive a regular expression back from one of your automata, demonstrating the equivalence in both directions.
- State the pumping lemma for regular languages and use it to prove that one specific clinical pattern is not regular — for example a note requiring equal numbers of opening and closing brackets — and relate the proof to your finding in Task 3.

Evidence Expected

Correct automata with full subset-construction and minimisation tables, a demonstrated two way equivalence with regular expressions, and a valid non-regularity proof.

TASK 5 – Inflectional Morphology 3 Marks

Guiding Enquiry

Which inflectional processes appear in clinical English, and how are surface forms systematically related to their stems?

Investigate and interpret:

- Define morpheme, stem, affix, inflection and paradigm, and separate inflection from derivation using the standard criteria of category preservation, productivity and position relative to other affixes.
- List every inflectional paradigm present in your corpus — noun number, verb tense and aspect, subject agreement, adjective comparison — and give the full paradigm for at least five clinical lexemes.
- State the orthographic rules governing English inflection — e-deletion, consonant doubling, y-to-i, the sibilant -es rule — precisely enough to be implemented, and give for each one a counter-example on which it over-generates.
- Identify the irregular forms in your corpus and explain why each must be listed in the lexicon rather than derived by rule.
- Measure the effect of inflection: report the type-to-lemma ratio of your corpus and the proportion of failures in your query set that inflection alone explains.

Evidence Expected

Precisely stated inflectional rules with paradigms, an irregular list, and a quantified measurement of the effect of inflection on retrieval.

TASK 6 – Derivational Morphology 4 Marks

Guiding Enquiry

How does derivation build the specialised clinical vocabulary, and why is it harder to handle than inflection?

Investigate and interpret:

- Define derivation and list at least fifteen derivational affixes attested in your corpus — for example -itis, -ectomy, -ology, -osis, -aemia, -ation, -able, -ive, nephro-, hyper-, hypo- , re-, un- — stating the category change each one causes.
- Build the derivational family of at least six clinical roots, showing the order in which affixes attach and identifying at least two orders that are impossible.
- Explain semantic non-compositionality with clinical examples in which the meaning of the derived word cannot be predicted from its parts, and state what this implies for a search system that stems aggressively.
- Compare the productivity of your affixes empirically: count types and tokens for each in the corpus and classify each affix as productive or fossilised, with the evidence for the classification.
- Decide whether derivational relatives should be conflated for search, citing at least two cases where conflation helps the clinician and two where it produces a clinically misleading match.

Evidence Expected

A documented derivational analysis with an affix inventory, productivity measurements and a reasoned position on conflation.

TASK 7 – Finite-State Morphological Parsing 5 Marks

Guiding Enquiry

How can a transducer relate a surface form to its stem and features, and run in both directions on the same machine?

Investigate and interpret:

- Define a finite-state transducer as a two-tape automaton and explain the lexical and surface tapes with one fully worked clinical example.

- Build your lexicon as an FST organised into continuation classes, covering at least three parts of speech together with the inflectional and derivational affixes of Tasks 5 and 6.
- Hand-trace the parsing of at least five surface forms, showing the state sequence, the symbols consumed and emitted on each tape, and the final analysis — for example fractures → fracture +N +PL.
- Run the same transducer in the generation direction, producing surface forms from stem plus-feature input, and show explicitly that parsing and generation use one machine and not two.
- Handle ambiguity: give at least two surface forms with more than one analysis, show the transducer returning all of them, and state how a downstream component would choose between them.

Evidence Expected

A working transducer specification with traced analyses in both directions and an explicit treatment of ambiguity.

TASK 8 – Orthographic Rules and Two-Level Morphology 4

Marks Guiding Enquiry

How are spelling changes at morpheme boundaries handled without listing every surface form in the lexicon?

Investigate and interpret:

- Explain why a lexicon transducer alone cannot produce correct surface spellings, using at least three examples in which plain concatenation yields the wrong string.
- Write at least five orthographic rules in the standard rewrite notation (a → b / c d), stating the context of each rule precisely.
- Implement each rule as a transducer and compose the rule transducers with the lexicon FST; draw the cascade and state the order dependencies between the rules.
- Contrast this sequential cascade with the parallel two-level (Koskenniemi) formulation, and explain the practical advantage each has over the other.
- Test the complete system on at least twenty forms chosen to exercise every rule, tabulate successes and failures, and account for every failure.

Evidence Expected

Correct rule transducers, a documented composition order, and a tested cascade with full error accounting.

TASK 9 – The Porter Stemmer 5 Marks

Guiding Enquiry

What does a rule-based stemmer achieve with no lexicon at all, and what does it cost in precision?

Investigate and interpret:

- State the algorithm precisely: the definition of consonant and vowel, the measure m of a word, and the conditions *v*, *d and *o, with the calculation worked through for at least six clinical words.
- Work through all five steps of the algorithm by hand for at least six words, showing which rule fires at each step and the intermediate form after each one.
- Contrast stemming with lemmatisation using your Task 7 transducer: stem and lemmatise the same 200 annotated forms and tabulate where the two agree and where they diverge.
- Quantify both error types with clinical examples — over-stemming, which conflates words that must stay distinct, and under-stemming, which fails to conflate related

words — and report the rate of each on your annotated set.

- Measure the retrieval effect: run your 40 queries three times, with no stemming, with the Porter stemmer and with full morphological analysis, and report precision, recall and F1 for each condition.

Evidence Expected
Correct hand-traces of the algorithm, a quantified error analysis, and a retrieval experiment comparing stemming against full analysis.

TASK 10 – Implement and Test the Pipeline 3 Marks

Guiding Enquiry

Can a working implementation reproduce the behaviour you derived

by hand? **Investigate and interpret:**

- Implement in Python the tokenizer, the entity patterns, the automata including the subset construction, the morphological transducer and the Porter stemmer, using the standard library and NLTK only.
- Run the complete pipeline over the corpus and report, for every stage, the number of items processed, matched or analysed, the number rejected, and the run-time.
- Include test cases for every automaton, at least one string that must be rejected per pattern, the ambiguous forms from Task 7 and the irregulars from Task 5, and check the output against your hand-computed results from Tasks 4, 7 and 9.
- Present the results in comparable tables and at least one plot — for example analysis coverage against lexicon size, or run-time against record length.

Evidence Expected

Working, documented code, a reproducible test procedure, and results tables and graphs.

TASK 11 – Compare, Recommend and Defend the Normalisation Design 2

Marks Guiding Enquiry

Which normalisation design would you deploy in the hospital, and what evidence supports your choice?

Investigate and interpret:

- Compare the three conflation strategies — no normalisation, Porter stemming and full finite-state morphological analysis — on retrieval quality, coverage, run-time and maintenance effort, using your own measured results.
- Recommend a design, stating its expected accuracy, its failure modes, and what a clinician would actually notice when it fails.
- Discuss the manual effort involved in pattern writing, lexicon and affix construction and annotation, and identify the point at which a statistical or learned approach becomes more economical than hand-built finite-state resources.

Evidence Expected

A justified recommendation supported by theory and measurements; choosing the highest recall alone is not sufficient.

DAY-4 DESIGN CHALLENGE

TASK 12 – Re-evaluate and Adapt the Design 8 Marks

Changed Requirement
The system must now ingest two new sources. The first is optical character recognition output from scanned handwritten forms, in which characters are routinely confused (0 for O, 1 for l, rn for m), words are broken across line endings, and diacritics are lost. The second is notes typed by staff in Romanised regional language mixed with English, for example “patient ku

fever irundhuchu, paracetamol 500mg kuduthen". At least 20% of incoming tokens are expected to be out of lexicon, misspelt, or not English at all.

Re-evaluate your original design under the changed conditions. You

may: **RETAIN → MODIFY → or REDESIGN**

your original solution.

A. Impact Analysis – 2 Marks
Guiding Enquiry

What exactly has changed, and which of your original assumptions no longer hold?

- Identify the properties of the new input that your patterns, automata and analyser cannot handle: character-level noise, words hyphenated across lines, out-of-lexicon stems, non-English morphology and token-level code-mixing.
- Identify the assumptions this breaks: clean typed input, a closed vocabulary, English only morphology, and exact matching by automata that accept or reject with nothing in between.
- Estimate how the noise rate will affect pattern precision and analyser coverage, reasoning from the error rates you measured in Tasks 2 and 7.

Evidence Expected

A clear analysis of how the new input exposes gaps in the original design.

B. Evaluation of the Original Design – 2 Marks

Guiding Enquiry

How would your recommended pipeline actually behave on the new input?

- Run or hand-trace at least three new records — one from OCR, one code-mixed, one both — through your original pipeline and record precisely what fails first and why.
- Measure the degradation: report pattern precision and recall, analyser coverage and retrieval F1 on the noisy sample against your clean-corpus baseline.
- Identify whether a different formalism — weighted automata, edit-distance matching, or a character n-gram language identifier — would handle the new input more economically than extending what you have.

Evidence Expected

A technically supported evaluation of the original design with traces or measurements.

C. Adaptation or Redesign – 2 Marks

Guiding Enquiry

Should the finite-state layer be extended, replaced or supplemented?

- Choose retain, modify or redesign — for example placing a weighted edit-distance transducer in front of the analyser, extending the lexicon with regional stems, adding token-level language identification, or falling back to stemming for out-of-lexicon tokens.
- Describe the revised design and state exactly what changes in the patterns, the automata, the lexicon and the rule cascade.
- Determine the effect on coverage, precision, run-time and the annotation effort needed to maintain the system.

Evidence Expected

A design response with clear linguistic and algorithmic reasoning and a quantified effect.

D. Evidence-Based Defence – 2 Marks

Guiding Enquiry

Why is your final decision the right one, given the evidence you generated?

- Use your pattern error analysis, automaton traces, coverage figures, retrieval measurements and effort estimates.
- Show that the decision follows logically from that evidence, and state its remaining limitations — including what a finite-state layer will never handle, however much it is extended.

Evidence Expected

A well-supported final decision integrating theory, experiments and the changed requirements.

MARK DISTRIBUTION

Task	**Marks**
Task 1 – Frame the Problem: Models, Algorithms and Levels of Analysis	3
Task 2 – Basic Regular Expression Patterns	4
Task 3 – Regular Expressions for Tokenisation and Normalisation	4
Task 4 – Finite State Automata: Design, Non-determinism and Equivalence	5
Task 5 – Inflectional Morphology	3
Task 6 – Derivational Morphology	4
Task 7 – Finite-State Morphological Parsing	5
Task 8 – Orthographic Rules and Two-Level Morphology	4
Task 9 – The Porter Stemmer	5
Task 10 – Implement and Test the Pipeline	3
Task 11 – Compare, Recommend and Defend the Normalisation Design	2
Subtotal	**42**

Task 12 – Day-4 Design Challenge	8
TOTAL	**50**

TASK 1 – Frame the Problem: Models, Algorithms and Levels of Analysis (3 Marks)

1.1 Levels of linguistic analysis

Level	Meaning	Hospital example / relevance
Phonology	Sound patterns in spoken language.	Not central to typed records; may matter later for speech transcription or OCR confusion modelling.
Morphology	Internal structure of words: stems and affixes.	fracture, fractures, fractured, fracturing; nephrectomy/nephrectomies. Required.
Syntax	How words form phrases and sentences.	"No evidence of infection" differs from "evidence of infection". Useful for context and negation.
Semantics	Meaning of words and expressions.	"renal" and "kidney" are related; dosage and lab values need meaning aware interpretation. Important for advanced search.
Pragmatics	Meaning in a specific situation or speaker intention.	Not solved by the finite-state layer.
Discourse	Meaning across sentences and document sections.	A discharge summary may refer back to a diagnosis introduced earlier. Useful for full clinical understanding, beyond the first version.

"Rule out pneumonia" is not a confirmed pneumonia diagnosis.

The first version requires tokenisation, structured-entity recognition and morphology. Syntax, semantics, pragmatics and discourse are not safely ignored for clinical interpretation; they can be deferred only because this assignment builds a retrieval-oriented finite-state foundation, not a diagnostic system.

1.2 Model families

Family	Best-fit task	Why it fits / why alternatives are weaker
State machines (FSA/FST)	Validate identifiers, dates and analyse word forms.	Finite-state transitions give explicit, testable acceptance rules. A probabilistic model is unnecessary for a strict ID format; a large hand written rule system is less convenient for state traversal.
Formal rule systems	Normalise known abbreviations and apply spelling rules.	Rules are transparent and deterministic. A state machine alone does not conveniently encode contextual domain substitutions; probability is undesirable for exact safety-critical replacement.
Probabilistic models	Rank ambiguous search results or suggest OCR corrections.	They estimate likely alternatives from data. A fixed FSA cannot rank alternatives; manually enumerated rules do not scale to noisy, uncertain text.

1.3 Ambiguity in clinical text

Lexical ambiguity: "cold" may mean low temperature or a common cold; context is needed.
Morphological ambiguity: "reading" can be a noun or a verb form; "lesions" is noun + plural.
Segmentation ambiguity: "500mg" may be one number-unit token or two tokens, depending on the tokenizer policy.

Abbreviation ambiguity: “MS” may mean multiple sclerosis, morphine sulfate or another local abbreviation. It must not be expanded without context.

1.4 Search and resource costs

Automaton recognition explores states and transitions; a DFA scans an input of length n in O(n) time and O(1) extra working memory when transitions are table-driven. NFA simulation may track multiple active states. Regex backtracking engines can consume more time and stack/memory on ambiguous patterns, so bounded patterns and timeouts are recommended. Lexicon construction, annotation and clinician relevance judgements consume expert effort; probabilistic correction additionally requires representative labelled data.

1.5 Proposed pipeline

Stage Input → output Formalism

1. Ingest PDF/OCR/typed note → raw text File extraction / OCR

2. Clean raw text → corrected character stream + provenance Unicode rules; cautious OCR candidates

3. Tokenise text → tokens Regex + maximal munch 4. Entity recognition tokens → IDs, dates, drugs, measurements Anchored regex + DFA validators 5. Normalise surface strings → canonical formats Deterministic rewrite rules

6. Morphological analysis word forms → stem + grammatical features Lexicon FST + orthographic rules 7. Search indexing canonical entities + stems → searchable index Inverted index

8. Query processing query → same normalised representation Same tokenizer and analyser 9. Retrieval query terms → ranked records Boolean retrieval / ranking; later probabilistic model

Output: searchable, auditable tokens and entity annotations linked to the original record offsets. The original text is always retained.

TASK 2 – Basic Regular Expression Patterns (4 Marks)

2.1 Operators and examples

Operator Meaning Clinical example

Concatenation A followed by B [0-9][0-9] matches two digits.

Disjunction | Either alternative mg|mcg matches either unit.

Kleene star * Zero or more [0-9]* allows optional digit sequence. Plus + One or more [0-9]+ matches one or more digits.

Optional ? Zero or one kg? matches k or kg; use (kg)? for optional kg. Character class [] One character from a set [ABC]

Range Continuous character interval [0-9], [A-Z]

Negated class [^] Any character except listed [^\s]+ is a non-whitespace run.

Anchors ^ $ Start and end of string ^PT[0-9]{6}$

Counters {m,n} Bounded repetitions [0-9]{2,4}

2.2 Twelve entity patterns

The expressions below use Python regular-expression syntax. Compile with re.IGNORECASE only where case-insensitivity is intended. For identifiers and dates, use fullmatch rather than searching inside arbitrary text.

Entity	Regex (Python raw string)	Positive example	Must not match
Patient identifier	\\bPT[0-9]{6}\\b	PT123456	Rejects wrong prefix, non-six-digit IDs and embedded longer IDs.
Admission date	\\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\\b	04/11/2026	Rejects impossible calendar dates; validate with datetime after regex.
Discharge date	\\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-9]{2}\\b	06-11-2026	Same syntax as admission; semantic date order checked separately.
Drug + strength	\\b[A-Za-z][A-Za-z-]{1,30}\\s+[0-9]+(?:\\.[0-9]+)?\\s*(?:mg|mcg|g|ml|mL)\\b	Paracetamol 500 mg	Rejects strength without drug name and unsupported units; dictionary validation needed.
Dosage frequency	\\b(?:OD|BD|TDS|QID|once daily|twice daily|every\\s+[0-9]+\\s*h(?:ours?)?)\\b	twice daily	Rejects unknown abbreviations; local policy may add terms.
Lab value + unit	\\b[A-Za-z][A-Za-z0-9-]*\\s*[:=]?\\s*[0-9]+(?:\\.[0-9]+)?\\s*(?:mg/dL|g/dL|mmol/L|10\\^?9/L|U/L)\\b	Hb 13.2 g/dL	Rejects unsupported units; test name/unit compatibility separately.
Blood pressure	\\b(?:[7-9][0-9]|1[0-9]{2}|2[0-4][0-9])\\s*/\\s(?:[4-9][0-9]|1[0-4][0-9])\\s*mmHg\\b	120/80 mmHg	Rejects out-of-range values in this simplified profile; clinical limits are configurable.
Symptom duration	\\b[0-9]+\\s*(?:hours?|days?|weeks?|months?)\\b	3 days	
Temperature	\\b(?:3[5-9]|4[0-2])(?:\\.[0-9])?\\s*°?C\\b	38.2 C	Rejects Fahrenheit and values outside configured range; add °F separately if required.
ICD code	\\b[A-TV-Z][0-9][A-Z0-9](?:\\.[A-Z0-9]{1,4})?\\b	S72.001A	Rejects malformed code shapes; validate against the relevant ICD release.
Ward and bed	\\bWard\\s+[A-Z0-9-]+\\s*,?\\s*Bed\\s+[A-Z0-9-]+\\b	Ward A2, Bed 14	Rejects missing ward or bed labels.
Contact number	\\b(?:\\+91[-]?)?[6-9][0-9]{9}\\b	+91 9876543210	Rejects short numbers and non-mobile prefixes under this India-specific rule.
Age	\\b(?:[1-9][0-9]?|1[01][0-9]|120)\\s*(?:years?|yrs?)\\b	19 years	Rejects zero, negative and >120; neonatal age needs a separate rule.

The brief lists twelve categories but admission and discharge dates are separate entities; this inventory therefore contains thirteen rows to cover every requested item.

2.3 Greedy and non-greedy matching

Sentence: “Give 5 mg after breakfast; give 10 mg at night.” The expression r"Give .* mg" is greedy and may match from the first “Give” through the last “mg”. The expression r"Give .*? mg" is non-greedy and stops at the first possible “mg”. Prefer a bounded expression such as r"Give\s+[0-9]+\s*mg" when extracting dosage instructions.

2.4 Precedence and grouping

Alternation has low precedence. The expression ^Ward|Bed$ means “starts with Ward” OR “ends with Bed”, not “Ward or Bed as the complete string”. Correct grouping is ^(?:Ward|Bed)$.

2.5 Evaluation protocol

For each entity type, manually label every occurrence in the 500-record corpus. Count TP (correctly detected), FP (incorrect detections) and FN (missed entities). Precision = TP/(TP+FP); recall = TP/(TP+FN). Regex errors commonly arise from OCR noise, punctuation variation, unseen abbreviations, impossible dates accepted by shape-only regex, and unit variants. No real corpus was supplied, so actual TP/FP/FN cannot be reported here.

Illustrative pattern	TP	FP	FN	Likely error cause
Patient ID	98	2	4	OCR O/0 confusion; missing prefix
Date	91	7	9	Invalid calendar dates; mixed formats
Drug + strength	86	8	14	Brand names and spacing variants
BP	42	3	5	Missing mmHg or OCR slash error
Contact	19	2	1	Extension/local number ambiguity

These values are demonstration-only and must not be represented as measured results.

TASK 3 – Regular Expressions for Tokenisation and Normalisation (4 Marks)

3.1 Token definition and tokenizer

A token is a maximal meaningful unit: a decimal number, a number-unit pair, a dotted abbreviation, a hyphenated clinical term, a word, or a punctuation mark. Keeping number unit pairs together preserves dosage information; keeping abbreviations together prevents “Dr.” from becoming two tokens.

```
TOKEN_RE = re.compile(
    r"""(?ix)
    \d+(?:\.\d+)?\s*(?:mg/dL|g/dL|mmol/L|mcg|mg|ml|mL|kg|cm|°C)\b
    |[A-Za-z](?:[A-Za-z]\.){1,5} # e.g. U.S., I.V.
    |[A-Za-z]+(?:-[A-Za-z]+)+ # e.g. follow-up
    |[A-Za-z]+(?:'[A-Za-z]+)?
    |\d+(?:\.\d+)?
    |[^\w\s]
    """)
```

3.2 Maximal-munch examples

Input	Whitespace split	Proposed tokenisation
Take 500mg daily.	Take | 500mg | daily.	Take | 500mg | daily | .
BP 120/80 mmHg	BP | 120/80 | mmHg	BP | 120 | / | 80 | mmHg (entity parser combines BP value)
No post-operative fever.	No | post-operative | fever.	No | post-operative | fever | .
Dr. A. Kumar	Dr. | A. | Kumar	Dr. | A. | Kumar

Maximal munch chooses the longest token accepted at the current character position. A production tokenizer should use ordered alternatives and preserve character offsets. The BP example intentionally tokenises components; the entity recogniser later groups them.

3.3 Abbreviation expansion and canonicalisation

Abbreviation	Canonical expansion
BP	blood pressure
HR	heart rate
RR	respiratory rate
Temp	temperature
SOB	shortness of breath
N/V	nausea and vomiting
HTN	hypertension
DM	diabetes mellitus
CKD	chronic kidney disease
AKI	acute kidney injury
MI	myocardial infarction
CVA	cerebrovascular accident
UTI	urinary tract infection
COPD	chronic obstructive pulmonary disease
CAD	coronary artery disease
IHD	ischaemic heart disease
CBC	complete blood count
LFT	liver function test
RFT	renal function test
ECG	electrocardiogram
CT	computed tomography
MRI	magnetic resonance imaging
IV	intravenous
IM	intramuscular
OD	once daily
BD	twice daily
TDS	three times daily
QID	four times daily
SOS	if required
PRN	as needed

Expansion is context-sensitive: “MS”, “OD” and “CT” have multiple meanings. Only unambiguous abbreviations should be expanded automatically. Store original text and the normalisation decision.

3.4 Example normalisation

Synthetic input record: “PT123456 adm 04/11/2026. BP:120/80 mmHg. Pt c/o fever x 2 days. Paracetamol 500mg BD.”
Normalised representation: “PT123456 admission_date=2026-11-04. blood pressure=120/80 mmHg. patient complains of fever duration=2 days. paracetamol 500 mg twice daily.”

3.5 Irreversible decisions and limits

Lowercasing removes original capitalisation, which may distinguish abbreviations or proper names. Retain the original string.
Expanding an ambiguous abbreviation can destroy the original uncertainty. Preserve the abbreviation and attach candidate expansions.
Removing punctuation can destroy decimal, range or section information. Preserve offsets and punctuation where clinically meaningful.
Ordinary regular expressions recognise regular languages and cannot, in their classical finite-state form, match arbitrarily nested balanced parentheses. A stack-based pushdown automaton is required for unbounded nesting.

TASK 4 – Finite-State Automata: Design, Non-determinism and Equivalence (5 Marks)

4.1 Formal definition

A finite automaton is a five-tuple M = (Q, Σ, δ, q_0, F), where Q is a finite set of states, Σ is the input alphabet, δ is the transition function, q_0 is the start state and F is the set of accepting states.

4.2 DFA validators

Four validators are proposed. State names and transition groups below are compact transition diagrams; “other” means every symbol not listed goes to a rejecting sink state. **Validator Transition diagram (text form) Transition table summary**

Patient ID PT + six digits q0 --P→ q1 --T→ q2 --digit→ q3 --digit→ q4 -- digit→ q5 --digit→ q6 --digit→ q7 --digit→ q8*q0:P→q1; q1:T→q2; q2:[0-9]→q3; q3:[0-9]→q4; q4:[0-9]→q5; q5:[0-9]→q6; q6:[0-9]→q7; q7:[0-9]→q8; all other→sink.

Age (1–120 years) q0 --1..9→ q1*; q1 --digit→ q2*; q2*: q0 --1→q3 -- 0→q4 --0→q5*q0:[1-9]→q1; q1:[0-9]→q2; q0:1→q3; q3:0→q4; q4:0→q5; all other→sink. Add suffix “ years” in a second literal chain.

Temperature 35.0–42.9 C q0 --3→q1 --5..9→q2 --.→q3 --digit→q4*; or 4→q5 --0..2→q6 --.→q3 q0:3→q1,4→q5; q1:[5-9]→q2; q5:[0-2]→q6; q2/q6:..→q3; q3:[0- 9]→q4; q4:C→q7*.

Blood pressure q0 --systolic digits→q1 --/→q2 --diastolic digits→q3 --space mmHg→q4* q0:[1-2]→q1; q1:[0-9]→q1; q1:/→q2; q2:[0-9]→q3; q3:[0-9]→q3; q3:space→q4; q4:mmHg→q5*.

Each DFA can be expanded into at least six explicit states by separating every required literal and digit position. In implementation, an explicit dead state ensures every transition is defined. A full transition table is represented by the transition summaries above; the Python validator in the appendix gives executable acceptance checks.

4.3 NFA for optional date separators

Target format: DD[-/]MM[-/]YYYY, where the separator may be “-” or “/”. An NFA branches after DD: one branch consumes “-” and the other consumes “/”, then both merge before MM. The two alternatives are convenient to represent nondeterministically. q0 --digit--> q1 --digit--> q2\nq2 -- - --> q3 --digit--> q4 ...\nq2 -- / --> q3 --digit--> q4 ...\nBoth branches merge at

the month/year suffix.
Accepted trace: 04/11/2026 follows the slash branch and reaches the final state. Rejected trace: 04x11/2026 has no transition on x after q2, so the active-state set becomes empty.

4.4 Subset construction and minimisation NFA

state subset Input "-" Input "/" Input digit

{q0} ∅ ∅ {q1} {q1} ∅ ∅ {q2} {q2} {q3} {q3} ∅

{q3} ∅ ∅ {q4} {q4} ∅ ∅ {q5} {q5} ∅ ∅ {q6} {q6} ∅ ∅ {q7} {q7} ∅ ∅ {q8} {q8} ∅ ∅ {q9} {q9} ∅ ∅ {q10*}

For a complete implementation, digit positions are expanded and the month/day ranges are checked by a semantic date validator. The two separator paths are equivalent after they merge; DFA minimisation merges equivalent suffix states. Exact state counts depend on whether digit classes and semantic calendar validation are included. A claim of a particular minimised count should be made only after running a minimisation algorithm on the fully expanded transition graph.

4.5 Thompson construction and equivalence

For expression (a|b)c, Thompson construction creates a split ε-state, an a-path and b-path that rejoin, followed by a c transition. The resulting NFA accepts exactly {ac, bc}. Conversely, state elimination on the corresponding automaton yields (a|b)c. Both directions denote the same language.

4.6 Pumping lemma

Pumping lemma: if L is regular, there is a pumping length p such that every $s \in L$ with $|s| \geq p$ can be written $s=xyz$, $|xy| \leq p$, $|y|>0$, and $xy^iz \in L$ for every $i \geq 0$. Consider $L=\{(^n)^n : n \geq 0\}$, strings with n opening and n closing parentheses. Choose $s=(^n)^n$ with $n \geq p$. Since $|xy| \leq p$, y contains only opening parentheses. Pumping i=0 removes at least one opening parenthesis but leaves all n closing parentheses, so the counts differ and the result is not in L. Contradiction; L is not regular. This proves the Task 3 limitation.

TASK 5 – Inflectional Morphology (3 Marks)

5.1 Definitions and distinction

A morpheme is the smallest unit carrying meaning or grammatical function. A stem is the base to which an affix attaches. An affix is a bound morpheme attached to a base. Inflection changes grammatical form without changing lexical category or core meaning. A paradigm is the set of inflected forms of a lexeme.
Inflection is generally productive, category-preserving and usually outermost (e.g., fracture + plural -s). Derivation creates a new lexeme or meaning, may change category, is less predictable and may attach closer to the root.

5.2 Clinical inflectional paradigms

Lexeme Noun number Verb forms Other forms fracture fracture / fractures fracture / fractures / fractured /

fracturing — lesion lesion / lesions — —

infect — infect / infects / infected / infecting infection; infectious (derivational)

treat — treat / treats / treated / treating treatments (derivational + plural)

swollen — swell / swells / swelled / swelling swollen (irregular participle)

Agreement in English verbs includes third-person singular -s (patient reports), while adjective comparison is rare in formal clinical notes but may occur: high/higher/highest, severe/more severe/most severe.

5.3 Orthographic rules

Rule Implementation condition Example Possible over-generation Drop final e before -ing If stem ends consonant+e, remove e before -ing treat → treating see → seeing does not drop both e letters. some casesOften retain silent e to preserve soft c/g manage → Keep e before -able in manageable Not universal: use → usable drops e.

Double final consonant CVC monosyllabic/stressed final syllable before vowel suffix admit → admitted visit → visited, not visitted.

y → i Consonant+y before -es/-ed biopsy → biopsies play → played; vowel+y keeps y. Add -es After s, x, z, ch, sh; often consonant+o diagnosis → diagnoses photo → photos, not photoes.

5.4 Irregular forms

Surface form	Lemma	Feature
children	child	N + PL
men	man	N + PL
feet	foot	N + PL
teeth	tooth	N + PL
mice	mouse	N + PL
was	be	V + PAST + 1/3SG
were	be	V + PAST
been	be	V + PARTICIPLE
is	be	V + PRES + 3SG
had	have	V + PAST/PART
ran	run	V + PAST
fell	fall	V + PAST
swollen	swell	V + PARTICIPLE
worse	bad	ADJ + COMP
best	good	ADJ + SUPERL

Irregular forms cannot be generated reliably by a single productive suffix rule; they are stored as explicit lexical entries or exception mappings.

5.5 Measurement

Type-to-lemma ratio = number of distinct surface word types ÷ number of distinct lemmas. An illustrative corpus might contain 8,000 surface types and 6,200 lemmas, giving 1.29. The inflection-only query failure rate is: queries rescued by inflectional matching ÷ total failed queries before morphology. For example, 12 of 40 failed queries rescued gives 30%. These are placeholders; calculate them from the actual corpus and query set.

TASK 6 – Derivational Morphology (4 Marks)

6.1 Derivational affix inventory

Affix Meaning / function Example Typical effect -itis inflammation arthritis N → N

-ectomy surgical removal nephrectomy N → N

-ology study/field neurology N → N

-osis condition/process fibrosis N → N

-aemia blood condition anaemia N → N

-ation process/result ventilation V → N

-ment result/process treatment V → N

-er agent provider V → N

-able capable of treatable V → Adj

-ive having property supportive V/N → Adj nephro- kidney nephrology N root compound cardio- heart cardiology N

root compound hyper- excessive/high hypertension Adj/N modification hypo- low/under hypoglycaemia Adj/N

modification re- again reassess V → V

un- not/reverse unresponsive Adj/V → Adj -al relating to renal N → Adj

-ic relating to cardiac N → Adj

-ous characterised by infectious N/V → Adj

-y state/quality respiratory?Not a reliable productive rule; list lexicalised forms.

6.2 Derivational families and order

Root Family / attachment order

infect infect → infect + -ion = infection; infection + -ous = infectious; un- + responsive = unresponsive (separate root). treat treat →

treat + -ment = treatment; treat + -able = treatable; treat + -ed = treated (inflection outside derivation). nephr nephr + -ectomy =

nephrectomy; nephr + -ology = nephrology; nephro- + -path-y = nephropathy.

cardi cardi + -ology = cardiology; cardi + -ac = cardiac; cardio- + -vascular = cardiovascular. fib fib + -rosis

= fibrosis; fibrous (lexicalised adjective); fibrotic (medical adjective).

ventil ventil + -ate = ventilate; ventilate + -ion = ventilation; ventilation + -s = ventilations.

Impossible or ill-formed orders include *-ectomy + nephro- (suffix before the root), and *treat + -ment + -able in the intended meaning “able to be treated” (the derivational base is treat, producing treatable; treatmentable is not standard).

6.3 Non-compositionality and search implications

“Inflammation” is not simply any generic action of “inflaming”; “shock” in medicine is a specific circulatory state, not merely a sudden surprise. “Discharge” can mean release from hospital or a bodily fluid. Aggressive stemming can conflate unrelated senses and should not replace a clinical lexicon.

6.4 Productivity and conflation decision

Affixes such as -ed, -s and -ing are highly productive inflectional markers. Clinical derivational endings such as -itis, -ectomy and -osis are productive within specialised terminology but have lexical restrictions. Forms such as anaemia and fibrosis may be lexicalised/fossilised. Count unique derived types and total tokens per affix to establish productivity empirically.

Conflation helps Why

fracture / fractures / fractured Retrieves plural and tense variants of the same clinical concept. nephrectomy / nephrectomies Retrieves singular and plural procedure mentions.

Potentially misleading conflation Why

infection / infectious Disease/process vs property; not always interchangeable. hypertension / hypotension Opposite clinical states; shared fragments must not merge them.

TASK 7 – Finite-State Morphological Parsing (5 Marks)

7.1 Finite-state transducer

A finite-state transducer (FST) is a finite-state machine with an input tape and an output tape. In analysis, the surface form is read on the input tape and a lexical stem plus grammatical tags are emitted on the output tape. In generation, the relation is traversed in reverse: lexical form and tags produce the surface spelling.
Example: surface “fractures” maps to lexical “fracture+N+PL”. The surface tape consumes f r-a-c-t-u-r-e-s; the lexical tape emits fracture followed by noun and plural tags.

7.2 Lexicon and continuation classes

Class Example stems Permitted continuations

NOUN fracture, lesion, kidney, infection, patient +SG, +PL; selected derivations such as -al, -itis, - ectomy

VERB fracture, treat, admit, infect, swell +BASE, +3SG, +PAST, +PART, +ING ADJECTIVE severe, high, low, responsive +POS, +COMP, +SUPER; selected derivations CLINICAL ROOT nephr, cardi, gastr, neur, hepat -ology, -ectomy, -itis, -osis

A full resource should contain at least 300 stems, 20 inflectional affix entries, 15 derivational affix entries and a separate irregular-form table. A compact teaching lexicon is shown here; scale it by importing a clinician-reviewed terminology list.

7.3 Hand traces

Surface Lexical analysis Consumed / emitted

fractures fracture +N +PL fracture stem; final s ↔ +PL

lesions lesion +N +PL lesion stem; final s ↔ +PL

treated treat +V +PAST treat stem; -ed ↔ +PAST

treating treat +V +ING treat stem; e deletion; -ing ↔ +ING nephrectomies nephrectomy +N +PL nephrectomy stem; y→ie spelling alternation before plural s

7.4 Generation in reverse

Input fracture+N+PL generates “fractures”; treat+V+PAST generates “treated”; treat+V+ING generates “treating”. The same lexical relation is traversed in reverse, not implemented as a separate unrelated grammar.

7.5 Ambiguity

“reading” → read+V+ING or reading+N+SG. A POS tagger or sentence context selects the likely analysis.
“treated” → treat+V+PAST or treat+V+PART. Both analyses may be retained until syntax/context resolves them.
The analyser should return all valid analyses with provenance. A downstream contextual tagger or clinician-facing interface chooses or displays alternatives; it should not silently discard ambiguity.

TASK 8 – Orthographic Rules and Two Level Morphology (4 Marks)

8.1 Why concatenation alone fails

Plain concatenation would produce *treate+ing instead of treating, *biopsy+s instead of biopsies, and *admit+ed instead of admitted. A lexicon FST must be composed with spelling rule transducers.

8.2 Rewrite rules

Rule Contextual rewrite Example Final e deletion e → 0 / consonant_+ing treat+ing → treating y to i y → i / consonant +es biopsy+s → biopsies Consonant doubling C → CC / stressed CVC_+ed/-ing admit+ed → admitted Sibilant plural +s → +es / sibilant diagnosis+s → diagnoses Soft c/g preservation e retained before suffix beginning a/o where required notice+able → noticeable

8.3 Cascade and two-level comparison

Cascade: Lexicon FST → morpheme-boundary/spelling-rule transducers → surface form. Rule ordering matters: identify the morpheme boundary first, apply y/e changes, then doubling and suffix realisation. A sequential cascade is easy to debug but may be order sensitive. Koskenniemi two-level morphology expresses lexical-to-surface correspondences in parallel constraints; it can be compact and avoids some ordering problems, but rule interaction and implementation can be harder to explain.

8.4 Test plan

Lexical input Expected surface Rule exercised Result treat+ING treating e deletion Pass admit+PAST admitted doubling Pass biopsy+PL biopsies y→i Pass diagnosis+PL diagnoses sibilant plural Pass notice+ABLE noticeable e retention Pass fracture+PL fractures regular plural Pass lesion+PL lesions regular

plural Pass

study+PAST studied y→i Pass play+PAST played vowel+y exception Pass visit+PAST visited no doubling Pass

The table is a hand-designed test suite, not a measured 20-form corpus test. A complete submission should extend it to at least twenty annotated forms and report each failure, including exceptions and lexicalised spellings.

TASK 9 – The Porter Stemmer (5 Marks)

9.1 Algorithm concepts

Porter stemming is a suffix-stripping algorithm, not a dictionary lemmatiser. In the Porter measure, m counts vowel-consonant sequences in the stem: for example, “trouble” is written [C][VC][C][V], giving m=2 under the conventional grouping. A vowel is a, e, i, o or u; y is treated contextually as a consonant when preceded by a vowel and as a vowel otherwise. Conditions include *v (stem contains a vowel), *d (stem ends in a double consonant) and *o (stem ends in consonant-vowel-consonant, with final w/x/y excluded).

9.2 Worked measure examples

Word VC grouping (simplified) m / condition

fracture fr + a + ct + u + r + em depends on final-e handling; suffix rules inspect the stem before suffix.

lesions l + e + s + io + n + sstem after -s is lesion; suffix -s is removed only in applicable step.

treated tr + ea + t + ed suffix -ed; stem “treat” contains vowel (*v true). treating tr + ea + t + ing suffix -ing; stem “treat” contains vowel (*v true). admission a + d + m + i + ss + ion suffix -ion conditions depend on preceding stem ending. renal r + e + n + a + l no common Porter suffix necessarily removed.

The Porter algorithm uses five ordered steps (Step 1a/1b/1c, Step 2, Step 3, Step 4 and Step 5 in the classic description). Exact intermediate outputs depend on the canonical algorithm implementation; NLTK’s PorterStemmer should be used for reproducible execution.

9.3 Illustrative stem traces

Input Likely Porter output Clinical caution

fractures fractur Plural suffix removed; not a true lemma. fractured fractur Tense suffix removed.

fracturing fractur Progressive suffix removed.

lesions lesion Plural removed.

treatments treatment Plural removed; derivational suffix remains. nephrectomies nephrectomi / implementation dependentMay not recover nephrectomy; spelling and suffix rules can under-stem.

9.4 Stemming versus lemmatisation

A stemmer may output a non-word stem such as “fractur”. A morphological analyser returns a valid lemma and features, such as fracture+N+PL. Lemmatisation uses lexical and grammatical information and is more precise but requires a maintained lexicon.

9.5 Error types and retrieval experiment

Over-stemming: unrelated words are reduced to the same stem, potentially merging clinically distinct concepts.
Under-stemming: related forms remain separate, such as nephrectomy and nephrectomies if the stemmer does not handle the spelling alternation.
For the 200 annotated forms, calculate over-stemming rate = incorrectly conflated forms ÷ 200; under-stemming rate = related forms not conflated ÷ 200. For 40 queries, compute precision, recall and F1 under (1) no stemming, (2) Porter and (3) FST analysis. F1 = 2PR/(P+R).

Illustrative condition Precision Recall F1 Interpretation No normalisation 0.91 0.62 0.74 Precise but misses variants. Porter stemmer 0.84 0.78 0.81 Improved recall; some over-stemming.

Finite-state analysis 0.93 0.86 0.89Best balance if lexicon coverage is adequate.

These metrics are hypothetical examples only. Do not submit them as experimental results without running the query set.

TASK 10 – Implement and Test the Pipeline (3 Marks)

10.1 Implementation plan

Use Python standard library regular expressions and data structures, plus NLTK PorterStemmer. A production FST may be represented with adjacency maps or implemented using a finite-state library only if the course permits it; the brief restricts the implementation to standard library and NLTK, so a compact dictionary-based transducer is suitable for the assignment prototype.

```
import re
from collections import defaultdict
from nltk.stem import PorterStemmer

TOKEN_RE = re.compile(
    r"\d+(?:\.\d+)?\s*(?:mg/dL|g/dL|mmol/L|mcg|mg|ml|mL|kg|cm)\b"
    r"|[A-Za-z]+(?:-[A-Za-z]+)+|[A-Za-z]+|\d+(?:\.\d+)?|[^\w\s]",
    re.I
)
PATTERNS = {
    "patient_id": re.compile(r"\bPT[0-9]{6}\b"),
    "date": re.compile(r"\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-
    9]{2}\b"), "drug_strength": re.compile(r"\b[A-Za-z][A-Za-z-]{1,30}\s+[0-9]+(?:\.[0-
9]+)?\s*(?:mg|mcg|g|ml)\b", re.I),
    "bp": re.compile(r"\b[0-9]{2,3}\s*/\s*[0-9]{2,3}\s*mmHg\b", re.I),
}
ABBREVIATIONS = {"bp":"blood pressure", "htn":"hypertension",
                 "dm":"diabetes mellitus", "bd":"twice daily",
                 "tds":"three times daily", "prn":"as needed"}
def tokenize(text):
    return TOKEN_RE.findall(text)
def normalise_token(token):
    key = token.lower()
    return ABBREVIATIONS.get(key, key)
```

```
def recognise(text):
    return {name: pattern.findall(text) for name, pattern in PATTERNS.items()}
def analyse(word, lexicon):
    # lexicon maps surface forms to one or more (lemma, tags) analyses
    return lexicon.get(word.lower(), [])
stemmer = PorterStemmer()
def porter(word):
    return stemmer.stem(word.lower())
def pipeline(text, lexicon):
    tokens = tokenize(text)
    norm = [normalise_token(t) for t in tokens]
    entities = recognise(text)
    analyses = {t: analyse(t, lexicon) for t in tokens if t.isalpha()}
    stems = [porter(t) for t in tokens if t.isalpha()]
    return {"tokens": tokens, "normalised": norm,
            "entities": entities, "analyses": analyses, "porter": stems}
```

10.2 Test cases

Test Input	Expected result
Valid ID PT123456	Accepted
Invalid ID PT12345X	Rejected
Valid date 04/11/2026	Regex accepted; calendar validated
Invalid date 39/19/2026	Rejected
Drug strength Paracetamol 500 mg	Detected
BP 120/80 mmHg	Detected
Inflection fractures	fracture+N+PL
Irregular children	child+N+PL
Ambiguous	treated past and participle analyses
OCR PTI23456	Flag for possible I/1 correction; never auto-change ID

10.3 Runtime and output reporting

Record for each stage: number of records/tokens processed, matches or analyses produced, rejected items and elapsed seconds using time.perf_counter(). Plot runtime against record length and analyser coverage against lexicon size. Since no corpus was supplied, no actual run-time, count or graph can honestly be reported. The implementation above is a reproducible starting point; populate the lexicon and run it against the documented corpus.

TASK 11 – Compare, Recommend and Defend the Normalisation Design (2 Marks)

Design	Retrieval quality	Coverage	Runtime	Maintenance
No normalisation	High precision, lower recall	Very high surface		

	acceptance; no conflation	Very low	Low	
Porter stemming	Higher recall, risk of false matches	Broad English coverage	Low	Low
Finite-state morphology	High precision and recall for covered vocabulary	Limited by lexicon and rules	Low to moderate	High initial expert effort

Recommendation: deploy a hybrid, auditable design. Use regex/DFA for structured entities, conservative abbreviation expansion, and finite-state morphology for known clinical vocabulary. Retain original text and offsets. Use Porter stemming only as a fallback for non critical general-English search terms, never as the sole normaliser for drug names, diagnoses, identifiers or measurements.

Expected failure modes include unseen terms, local abbreviations, OCR substitutions, misspellings and semantic ambiguity. Clinicians may notice a relevant record is missed or an unrelated record is returned. Log these cases and route uncertain medication/identity matches for review. A statistical approach becomes more economical when the volume and diversity of noisy text make manual lexicon expansion and rule maintenance more expensive than collecting representative labelled data.

TASK 12 – DAY-4 DESIGN CHALLENGE: Re-evaluate and Adapt the Design (8 Marks)

A. Impact Analysis (2 Marks)

The new sources introduce OCR character noise, line-break hyphenation, lost diacritics, Romanised regional language, code-mixing and an estimated 20% out-of lexicon/misspelt/non-English token rate. The original design assumed clean typed English, a mostly closed vocabulary and exact accept/reject matching. These assumptions no longer hold.

Character noise: O/0, I/l/1 and rn/m substitutions cause exact identifiers and drug strings to fail.

Line-break hyphenation: “post-\noperative” may be split into two fragments or incorrectly joined.

Out-of-lexicon stems: the FST returns no analysis for unfamiliar forms. Code-mixing: Tamil words written in Roman script require language identification and regional-language morphology.

If 20% of tokens are affected, a closed English lexicon can lose at least a comparable share of token coverage in the worst case. Entity precision may also fall because fuzzy correction can create false candidates. Actual degradation must be measured on a labelled noisy sample.

B. Evaluation of the Original Design (2 Marks)

Record type	Synthetic example	First failure in original pipeline
Code-mixed	“patient ku fever irundhuchu, paracetamol 500mg kuduthen”	English-only morphology cannot analyse ku, irundhuchu or kuduthen; abbreviation/context may remain unresolved.
	“PTI23456 post-\noperat1ve fever 2 naal”	ID mismatch, broken word, digit substitution and Tamil “naal” are not handled.

A useful comparison sample should contain at least 100 noisy records, with gold labels for entities, tokens and morphology. Measure pattern precision/recall, FST coverage and query F1 against the clean baseline. Candidate alternatives: weighted finite-state transducers (rank plausible OCR corrections), bounded edit-distance matching (find near-miss words), and character n-gram language identification (classify English vs Romanised regional language).

C. Adaptation or Redesign (2 Marks)

Decision: MODIFY the original design into a layered hybrid pipeline rather than discard the finite-state layer.
OCR-aware preprocessor: preserve raw OCR; create candidate corrections for common confusions (O/0, I/l/1, rn/m) with confidence scores. Do not automatically alter patient IDs or medication strengths.
Line-break repair: join a trailing hyphen plus next-line continuation only when the resulting word is in a reviewed lexicon or has a high-confidence candidate.
Token-level language identification: use character n-grams and a curated list of common Romanised Tamil clinical phrases. Mark uncertain tokens rather than forcing English analysis.
Lexicon extension: add clinician-reviewed regional-language stems, transliteration variants and common code-mixed phrases; maintain source and confidence metadata. Weighted matching: place bounded edit-distance or weighted FST correction before morphological analysis for low-risk words. Keep strict exact matching for identifiers and medication strengths.
Fallback: use Porter stemming only for English out-of-lexicon non-critical terms; retain the original token and mark it as unanalysed.

D. Evidence-Based Defence (2 Marks)

This modification preserves the original strengths—deterministic entity patterns, finite-state validation and explainable morphology—while adding tolerance where the changed input requires it. It should increase analyser coverage and recall on noisy notes, at the cost of additional runtime, candidate ambiguity and annotation effort. Precision is protected by confidence thresholds, domain-specific validation and human review for high-risk fields.

Dimension	Expected direction	Control / evidence required
Coverage	Increase	Measure known + OOV token analysis on labelled noisy set.
Recall	Increase	Evaluate 40+ clinician queries on noisy records.
Precision	May decrease if fuzzy correction is unconstrained	Threshold candidate scores; exact validation for IDs/drugs.
Runtime	Increase moderately	Benchmark per 1,000 tokens and record length.
Maintenance	Increase	Versioned lexicon, annotation guide and clinician review.

Finite-state methods will still not solve unrestricted semantic interpretation, clinical reasoning, discourse reference, sarcasm, implicit negation or arbitrary nested structures. They provide a reliable normalisation layer, not a complete understanding of medical language.

APPENDIX A – Python reference implementation

The following compact example demonstrates the key regular-expression and Porter components. It should be expanded with the full pattern inventory, a 300+ stem lexicon, explicit FST transition tables, 200 annotated forms and the evaluation corpus before claiming

full compliance with the assignment constraints.

```
import re
import time
from nltk.stem import PorterStemmer

PATTERNS = {
    "patient_id": re.compile(r"\bPT[0-9]{6}\b"),
    "date": re.compile(r"\b(?:0[1-9]|[12][0-9]|3[01])[-/](?:0[1-9]|1[0-2])[-/](?:19|20)[0-
    9]{2}\b"), "drug_strength": re.compile(r"\b[A-Za-z][A-Za-z-]{1,30}\s+[0-9]+(?:\.[0-
9]+)?\s*(?:mg|mcg|g|ml)\b", re.I),
    "frequency": re.compile(r"\b(?:OD|BD|TDS|QID|once daily|twice daily)\b", re.I),
    "bp": re.compile(r"\b[0-9]{2,3}\s*/\s*[0-9]{2,3}\s*mmHg\b", re.I),
    "temperature": re.compile(r"\b[0-9]{2}(?:\.[0-9])?\s*°?C\b", re.I),
    "age": re.compile(r"\b(?:[1-9][0-9]?|1[01][0-9]|120)\s*(?:years?|yrs?)\b", re.I),
"duration": re.compile(r"\b[0-9]+\s*(?:hours?|days?|weeks?|months?)\b", re.I), }
TOKEN = re.compile(
    r"\d+(?:\.\d+)?\s*(?:mg/dL|g/dL|mmol/L|mcg|mg|ml|mL|kg|cm)\b"
    r"|[A-Za-z]+(?:-[A-Za-z]+)+|[A-Za-z]+|\d+(?:\.\d+)?|[^\w\s]", re.I
)
ABBR = {"bp":"blood pressure", "htn":"hypertension",
        "dm":"diabetes mellitus", "bd":"twice daily",
        "tds":"three times daily", "prn":"as needed"}
porter = PorterStemmer()

def run_pipeline(text):
    start = time.perf_counter()
    tokens = TOKEN.findall(text)
    entities = {k: v.findall(text) for k, v in PATTERNS.items()}
    normalised = [ABBR.get(t.lower(), t.lower()) for t in tokens]
    stems = [porter.stem(t.lower()) for t in tokens if t.isalpha()]
    return {
        "tokens": tokens, "normalised": normalised,
        "entities": entities, "porter_stems": stems,
        "runtime_seconds": time.perf_counter() - start
    }

sample = ("PT123456 admitted 04/11/2026. BP 120/80 mmHg. "
          "Paracetamol 500 mg BD for 3 days.")
print(run_pipeline(sample))
```

Conclusion

A hospital search normalisation layer should combine regular expressions, finite-state automata, morphological parsing and conservative stemming. Regular expressions and DFAs are suitable for structured formats; FSTs connect surface words with lemmas and grammatical features; Porter stemming offers broad but less precise conflation. The Day-4 OCR and code-mixing requirement makes a hybrid design necessary. The final deployment decision must be supported by measurements from the required corpus and clinician query set, not by illustrative figures.